XAML placeholder conventions in Windows Runtime reference

If you've examined any of the **Syntax** section of reference topics for Windows Runtime APIs that can use XAML, you've probably seen that the syntax includes quite a few placeholders. XAML syntax is different than the C#, Microsoft Visual Basic or Visual C++ component extensions (C++/CX) syntax because the XAML syntax is a usage syntax. It's hinting at your eventual usage in your own XAML files, but without being over-prescriptive about the values you can use. So usually the usage describes a type of grammar that mixes literals and placeholders, and defines some of the placeholders in the **XAML Values** section.

When you see type names / element names in a XAML syntax for a property, the name that's shown is for the type that originally defines the property. But Windows Runtime XAML supports a class inheritance model for the DependencyObject-based classes. So you can often use an attribute on a class that's not literally the defining class, but instead derives from a class that first defined the property/attribute. For example, you can set Visibility as an attribute on any UIElement derived class using a deep inheritance. For example: `<Button Visibility="Visible" />`. So don't take the element name shown in any XAML usage syntax too literally; the syntax may be viable for elements representing that class, and also elements that represent a derived class. In cases where it's rare or impossible for the type shown as the defining element to be in a real-world usage, that type name is deliberately lowercased in the syntax. For example, the syntax you see for **UIElement.Visibility** is :

syntax

```
<uiElement Visibility="Visible"/>
-or-
<uiElement Visibility="Collapsed"/>
```

Many XAML syntax sections include placeholders in the "Usage" that are then defined in a **XAML Values** section that's directly under the **Syntax** section.

XAML usage sections also use various generalized placeholders. These placeholders aren't redefined every time in **XAML Values**, because you'll guess or eventually learn what they represent. We think most readers would get tired of seeing them in **XAML Values** again and again so we left them out of the definitions. For reference, here's a list of some of these placeholders and what they mean in a general sense:

- *object*: theoretically any object value, but often practically limited to certain types of objects such as a string-or-object choice, and you should check the Remarks on

the reference page for more info.

- *object property*: *object property* in combination is used for cases where the syntax being shown is the syntax for a type that can be used as an attribute value for many properties. For example, the **Xaml Attribute Usage** shown for **Brush** includes: *<object property="predefinedColorName"/>*
- *eventhandler*: This appears as the attribute value for every XAML syntax shown for an event attribute. What you're supplying here is the function name for an event handler function. That function must be defined in the code-behind for the XAML page. At the programming level, that function must match the delegate signature of the event that you're handling, or your app code won't compile. But that's really a programming consideration, not a XAML consideration, so we don't try to hint anything about the delegate type in the XAML syntax. If you want to know which delegate you should be implementing for an event, that's in the **Event information** section of the reference topic for the event, in a table row that's labeled **Delegate**.
- *enumMemberName*: shown in attribute syntax for all enumerations. There's a similar placeholder for properties that use an enumeration value, but it usually prefixes the placeholder with a hint of the enumeration's name. For example, the syntax shown for **FrameworkElement.FlowDirection** is *<frameworkElement***FlowDirection**="*flowDirectionMemberName*"/>. If you're on one of those property reference pages, click the link to the enumeration type that appears in the **Property Value** section, next to the text **Type:**. For the attribute value of a property that uses that enumeration, you can use any string that is listed in the **Member** column of the **Members** list.
- *double*, *int*, *string*, *bool*: These are primitive types known to the XAML language. If you're programming using C# or Visual Basic, these types are projected to Microsoft .NET equivalent types such as **Double**, **Int32**, **String** and **Boolean**, and you can use any members on those .NET types when you work with your XAML-defined values in .NET code-behind. If you're programming using C++/CX, you'll use the C++ primitive types but you can also consider these equivalent to types defined by the **Platform** namespace, for example **Platform::String**. There will sometimes be additional value restrictions for particular properties. But you'll usually see these noted in a **Property value** section or Remarks section and not in a XAML section, because any such restrictions apply both to code usages and XAML usages.

Tips and tricks, notes on style

- Markup extensions in general are described in the main XAML overview. But the markup extension that most impacts the guidance given in this topic is the StaticResource markup extension (and related ThemeResource). The function of the

StaticResource markup extension is to enable factoring your XAML into reusable resources that come from a XAML **ResourceDictionary**. You almost always define control templates and related styles in a **ResourceDictionary**. You often define the smaller parts of a control template definition or app-specific style in a **ResourceDictionary** too, for example a **SolidColorBrush** for a color that your app uses more than once for different parts of UI. By using a StaticResource, any property that would otherwise require a property element usage to set can now be set in attribute syntax. But the benefits of factoring XAML for reuse go beyond just simplifying the page-level syntax. For more info, see ResourceDictionary and XAML resource references.

- You'll see several different conventions for how white space and line feeds are applied in XAML examples. In particular, there are different conventions for how to break up object elements that have a lot of different attributes set. That's just a matter of style. The Visual Studio XML editor applies some default style rules when you edit XAML, but you can change these in the settings. There are a small number of cases where the white space in a XAML file is considered significant; for more info see XAML and whitespace.

Related topics

- XAML overview
- XAML namespaces and namespace mapping
- ResourceDictionary and XAML resource references

Property-path syntax

Article • 10/20/2022

You can use the PropertyPath class and the string syntax to instantiate a **PropertyPath** value either in XAML or in code. **PropertyPath** values are used by data binding. A similar syntax is used for targeting storyboarded animations. For both scenarios, a property path describes a traversal of one or more object-property relationships that eventually resolve to a single property.

You can set a property path string directly to an attribute in XAML. You can use the same string syntax to construct a PropertyPath that sets a Binding in code, or to set an animation target in code using SetTargetProperty. There are two distinct feature areas in the Windows Runtime that use a property path: data binding, and animation targeting. Animation targeting doesn't create underlying Property-path syntax values in the Windows Runtime implementation, it keeps the info as a string, but the concepts of object-property traversal are very similar. Data binding and animation targeting each evaluate a property path slightly differently, so we describe property path syntax separately for each.

Property path for objects in data binding

In Windows Runtime, you can bind to the target value of any dependency property. The source property value for a data binding doesn't have to be a dependency property; it can be a property on a business object (for example a class written in a Microsoft .NET language or C++). Or, the source object for the binding value can be an existing dependency object already defined by the app. The source can be referenced either by a simple property name, or by a traversal of the object-property relationships in the object graph of the business object.

You can bind to an individual property value, or you can bind to a target property that holds lists or collections. If your source is a collection, or if the path specifies a collection property, the data-binding engine matches the collection items of the source to the binding target, resulting in behavior such as populating a ListBox with a list of items from a data source collection without needing to anticipate the specific items in that collection.

Traversing an object graph

The element of the syntax that denotes the traversal of an object-property relationship in an object graph is the dot (.) character. Each dot in a property path string indicates a

division between an object (left side of the dot) and a property of that object (right side of the dot). The string is evaluated left-to-right, which enables stepping through multiple object-property relationships. Let's look at an example:

syntax

```
"{Binding Path=Customer.Address.StreetAddress1}"
```

Here's how this path is evaluated:

1. The data context object (or a **Source** specified by the same **Binding**) is searched for a property named "Customer".
2. The object that is the value of the "Customer" property is searched for a property named "Address".
3. The object that is the value of the "Address" property is searched for a property named "StreetAddress1".

At each of these steps, the value is treated as an object. The type of the result is checked only when the binding is applied to a specific property. This example would fail if "Address" were just a string value that didn't expose what part of the string was the street address. Typically, the binding is pointing to the specific nested property values of a business object that has a known and deliberate information structure.

Rules for the properties in a data-binding property path

- All properties referenced by a property path must be public in the source business object.
- The end property (the property that is the last named property in the path) must be public and must be mutable – you can't bind to static values.
- The end property must be read/write if this path is used as the **Path** information for a two-way binding.

Indexers

A property path for data-binding can include references to indexed properties. This enables binding to ordered lists/vectors, or to dictionaries/maps. Use square brackets "[]" characters to indicate an indexed property. The contents of these brackets can be either an integer (for ordered list) or an unquoted string (for dictionaries). You can also bind to a dictionary where the key is an integer. You can use different indexed properties in the same path with a dot separating the object-property.

For example, consider a business object where there is a list of "Teams" (ordered list), each of which has a dictionary of "Players" where each player is keyed by last name. An example property path to a specific player on the second team is: "Teams[1].Players[Smith]". (You use 1 to indicate the second item in "Teams" because the list is zero-indexed.)

Note Indexing support for C++ data sources is limited; see Data binding in depth.

Attached properties

Property paths can include references to attached properties. Because the identifying name of an attached property already includes a dot, you must enclose any attached property name within parentheses so that the dot isn't treated as an object-property step. For example, the string to specify that you want to use **Canvas.ZIndex** as a binding path is "(Canvas.ZIndex)". For more info on attached properties see Attached properties overview.

Combining property path syntax

You can combine various elements of property path syntax in a single string. For example, you can define a property path that references an indexed attached property, if your data source had such a property.

Debugging a binding property path

Because a property path is interpreted by a binding engine and relies on info that may be present only at run-time, you must often debug a property path for binding without being able to rely on conventional design-time or compile-time support in the development tools. In many cases the run-time result of failing to resolve a property path is a blank value with no error, because that is the by-design fallback behavior of binding resolution. Fortunately, Microsoft Visual Studio provides a debug output mode that can isolate which part of a property path that's specifying a binding source failed to resolve. For more info on using this development tool feature, see "Debugging" section of Data binding in depth.

Property path for animation targeting

Animations rely on targeting a dependency property where storyboarded values are applied when the animation runs. To identify the object where the property to be animated exists, the animation targets an element by name (x:Name attribute). It is often

necessary to define a property path that starts with the object identified as the **Storyboard.TargetName**, and ends with the particular dependency property value where the animation should apply. That property path is used as the value for **Storyboard.TargetProperty**.

For more info on the how to define animations in XAML, see Storyboarded animations.

Simple targeting

If you are animating a property that exists on the targeted object itself, and that property's type can have an animation applied directly to it (rather than to a sub-property of a property's value) then you can simply name the property being animated without any further qualification. For example, if you are targeting a **Shape** subclass such as **Rectangle**, and you are applying an animated **Color** to the **Fill** property, your property path can be "Fill".

Indirect property targeting

You can animate a property that is a sub-property of the target object. In other words, if there's a property of the target object that's an object itself, and that object has properties, you must define a property path that explains how to step through that object-property relationship. Whenever you are specifying an object where you want to animate a sub-property, you enclose the property name in parentheses, and you specify the property in *typename.propertyname* format. For example, to specify that you want the object value of a target object's **RenderTransform** property, you specify "(UIElement.RenderTransform)" as the first step in the property path. This isn't yet a complete path, because there are no animations that can apply to a **Transform** value directly. So for this example, you now complete the property path so that the end property is a property of a **Transform** subclass that can be animated by a **Double** value: "(UIElement.RenderTransform).(CompositeTransform.TranslateX)"

Specifying a particular child in a collection

To specify a child item in a collection property, you can use a numeric indexer. Use square brackets "[]" characters around the integer index value. You can reference only ordered lists, not dictionaries. Because a collection isn't a value that can be animated, an indexer usage can never be the end property in a property path.

For example, to specify that you want to animate the first color stop color in a **LinearGradientBrush** that is applied to a control's **Background** property, this is the

property path: "(Control.Background).(GradientBrush.GradientStops)[0].(GradientStop.Color)". Note how the indexer is not the last step in the path, and that the last step particularly must reference the GradientStop.Color property of item 0 in the collection to apply a Color animated value to it.

Animating an attached property

It isn't a common scenario, but it is possible to animate an attached property, so long as that attached property has a property value that matches an animation type. Because the identifying name of an attached property already includes a dot, you must enclose any attached property name within parentheses so that the dot isn't treated as an object-property step. For example, the string to specify that you want to animate the Grid.Row attached property on an object, use the property path "(Grid.Row)".

Note For this example, the value of Grid.Row is an **Int32** property type. so you can't animate it with a **Double** animation. Instead, you'd define an ObjectAnimationUsingKeyFrames that has DiscreteObjectKeyFrame components, where the ObjectKeyFrame.Value is set to an integer such as "0" or "1".

Rules for the properties in an animation targeting property path

- The assumed starting point of the property path is the object identified by a Storyboard.TargetName.
- All objects and properties referenced along the property path must be public.
- The end property (the property that is the last named property in the path) must be public, be read-write, and be a dependency property.
- The end property must have a property type that is able to be animated by one of the broad classes of animation types (Color animations, **Double** animations, Point animations, ObjectAnimationUsingKeyFrames).

The PropertyPath class

The PropertyPath class is the underlying property type of Binding.Path for the binding scenario.

Most of the time, you can apply a PropertyPath in XAML without using any code at all. But in some cases you may want to define a **PropertyPath** object using code and assign it to a property at run-time.

PropertyPath has a **PropertyPath(String)** constructor, and doesn't have a default constructor. The string you pass to this constructor is a string that's defined using the property path syntax as we explained earlier. This is also the same string you'd use to assign **Path** as a XAML attribute. The only other API of the **PropertyPath** class is the **Path** property, which is read-only. You could use this property as the construction string for another **PropertyPath** instance.

Related topics

- Data binding in depth
- Storyboarded animations
- {Binding} markup extension
- **PropertyPath**
- **Binding**
- **Binding constructor**
- **DataContext**

Move and draw commands syntax

Article • 10/20/2022

Learn about the move and draw commands (a mini-language) that you can use to specify path geometries as a XAML attribute value. Move and draw commands are used by many design and graphics tools that can output a vector graphic or shape, as a serialization and interchange format.

Properties that use move and draw command strings

The move and draw command syntax is supported by an internal type converter for XAML, which parses the commands and produces a run-time graphics representation. This representation is basically a finished set of vectors that is ready for presentation. The vectors themselves don't complete the presentation details; you'll still need to set other values on the elements. For a **Path** object you also need values for **Fill**, **Stroke**, and other properties, and then that **Path** must be connected to the visual tree somehow. For a **PathIcon** object, set the **Foreground** property.

There are two properties in the Windows Runtime that can use a string representing move and draw commands: **Path.Data** and **PathIcon.Data**. If you set one of these properties by specifying move and draw commands, you typically set it as a XAML attribute value along with other required attributes of that element. Without getting into the specifics, here's what that looks like:

XML

```
<Path x:Name="Arrow" Fill="White" Height="11" Width="9.67"
  Data="M4.12,0 L9.67,5.47 L4.12,10.94 L0,10.88 L5.56,5.47 L0,0.06" />
```

Using move and draw commands versus using a PathGeometry

For Windows Runtime XAML, the move and draw commands produce a **PathGeometry** with a single **PathFigure** object with a **Figures** property value. Each draw command produces a **PathSegment** derived class in that single **PathFigure**'s **Segments** collection, the move command changes the **StartPoint**, and existence of a close command sets **IsClosed** to **true**. You can navigate this structure as an object model if you examine the **Data** values at run time.

The basic syntax

The syntax for move and draw commands can be summarized like this:

1. Start with an optional fill rule. Typically you specify this only if you don't want the **EvenOdd** default. (More about **EvenOdd** later.)
2. Specify exactly one move command.
3. Specify one or more draw commands.
4. Specify a close command. You can omit a close command , but that would leave your figure open (that's uncommon).

General rules of this syntax are:

- Each command is represented by exactly one letter.
- That letter can be upper-case or lower-case. Case matters, as we'll describe.
- Each command except the close command is typically followed by one or more numbers.
- If more than one number for a command, separate with a comma or space.

[*fillRule*] *moveCommand drawCommand* [*drawCommand**] [*closeCommand*]

Many of the draw commands use points, where you provide an *x,y* value. Whenever you see a **points* placeholder you can assume you're giving two decimal values for the *x,y* value of a point.

White space can often be omitted when the result is not ambiguous. You can in fact omit all white space if you use commas as your separator for all number sets (points and size). For example, this usage is legal: `F1M0,58L2,56L6,60L13,51L15,53L6,64z`. But it's more typical to include white space between commands for clarity.

Don't use commas as the decimal point for decimal numbers; the command string is interpreted by XAML and doesn't account for culture-specific number-formatting conventions that differ from those used in the **en-us** locale.

Syntax specifics

Fill rule

There are two possible values for the optional fill rule: **F0** or **F1**. (The **F** is always uppercase.) **F0** is the default value; it produces **EvenOdd** fill behavior, so you don't typically specify it. Use **F1** to get the **Nonzero** fill behavior. These fill values align with the values of the **FillRule** enumeration.

Move command

Specifies the start point of a new figure.

Syntax
`M` *startPoint* - or - `m` *startPoint*

Term	Description
startPoint	Point The start point of a new figure.

An uppercase **M** indicates that *startPoint* is an absolute coordinate; a lowercase **m** indicates that *startPoint* is an offset to the previous point, or (0,0) if there was no previous point.

Note It's legal to specify multiple points after the move command. A line is drawn to those points as if you specified the line command. However that's not a recommended style; use the dedicated line command instead.

Draw commands

A draw command can consist of several shape commands: line, horizontal line, vertical line, cubic Bezier curve, quadratic Bezier curve, smooth cubic Bezier curve, smooth quadratic Bezier curve, and elliptical arc.

For all draw commands, case matters. Uppercase letters denote absolute coordinates and lowercase letters denote coordinates relative to the previous command.

The control points for a segment are relative to the end point of the preceding segment. When sequentially entering more than one command of the same type, you can omit the duplicate command entry. For example, `L 100,200 300,400` is equivalent to `L 100,200 L 300,400`.

Line command

Creates a straight line between the current point and the specified end point. `l 20 30` and `L 20,30` are examples of valid line commands. Defines the equivalent of a LineGeometry object.

Syntax

Syntax
`L` *endPoint* - or - `l` *endPoint*

Term	Description
endPoint	Point The end point of the line.

Horizontal line command

Creates a horizontal line between the current point and the specified x-coordinate. `H 90` is an example of a valid horizontal line command.

Syntax
`H` *x* - or - `h` *x*

Term	Description
x	Double The x-coordinate of the end point of the line.

Vertical line command

Creates a vertical line between the current point and the specified y-coordinate. `v 90` is an example of a valid vertical line command.

Syntax
`V` *y* - or - `v` *y*

Term	Description
y	Double The y-coordinate of the end point of the line.

Cubic Bézier curve command

Creates a cubic Bézier curve between the current point and the specified end point by using the two specified control points (*controlPoint1* and *controlPoint2*). `C 100,200 200,400 300,200` is an example of a valid curve command. Defines the equivalent of a PathGeometry object with a BezierSegment object.

Syntax
`C` *controlPoint1 controlPoint2 endPoint* - or - `c` *controlPoint1 controlPoint2 endPoint*

Term	Description
controlPoint1	Point The first control point of the curve, which determines the starting tangent of the curve.
controlPoint2	Point The second control point of the curve, which determines the ending tangent of the curve.
endPoint	Point The point to which the curve is drawn.

Quadratic Bézier curve command

Creates a quadratic Bézier curve between the current point and the specified end point by using the specified control point (*controlPoint*). `q 100,200 300,200` is an example of a valid quadratic Bézier curve command. Defines the equivalent of a PathGeometry with a QuadraticBezierSegment.

Syntax
`Q` *controlPoint endPoint* - or - `q` *controlPoint endPoint*

Term	Description
controlPoint	Point The control point of the curve, which determines the starting and ending tangents of the curve.
endPoint	Point The point to which the curve is drawn.

Smooth cubic Bézier curve command

Creates a cubic Bézier curve between the current point and the specified end point. The first control point is assumed to be the reflection of the second control point of the previous command relative to the current point. If there is no previous command or if the previous command was not a cubic Bézier curve command or a smooth cubic Bézier curve command, assume the first control point is coincident with the current point. The second control point—the control point for the end of the curve—is specified by *controlPoint2*. For example, `S 100,200 200,300` is a valid smooth cubic Bézier curve command. This command defines the equivalent of a PathGeometry with a BezierSegment where there was preceding curve segment.

Syntax
`S` *controlPoint2 endPoint* - or - `s` *controlPoint2 endPoint*

Term	Description
controlPoint2	Point The control point of the curve, which determines the ending tangent of the curve.
endPoint	Point The point to which the curve is drawn.

Smooth quadratic Bézier curve command

Creates a quadratic Bézier curve between the current point and the specified end point. The control point is assumed to be the reflection of the control point of the previous command relative to the current point. If there is no previous command or if the previous command was not a quadratic Bézier curve command or a smooth quadratic Bézier curve command, the control point is coincident with the current point. This command defines the equivalent of a PathGeometry with a QuadraticBezierSegment where there was preceding curve segment.

Syntax
`T` *controlPoint endPoint* - or - `t` *controlPoint endPoint*

Term	Description

Term	Description
controlPoint	Point The control point of the curve, which determines the starting and tangent of the curve.
endPoint	Point The point to which the curve is drawn.

Elliptical arc command

Creates an elliptical arc between the current point and the specified end point. Defines the equivalent of a PathGeometry with an ArcSegment.

Syntax
`A` *size rotationAngle isLargeArcFlag sweepDirectionFlag endPoint* - or - `a` *size rotationAngle isLargeArcFlag sweepDirectionFlag endPoint*

Term	Description
size	Size The x-radius and y-radius of the arc.
rotationAngle	Double The rotation of the ellipse, in degrees.
isLargeArcFlag	Set to 1 if the angle of the arc should be 180 degrees or greater; otherwise, set to 0.
sweepDirectionFlag	Set to 1 if the arc is drawn in a positive-angle direction; otherwise, set to 0.
endPoint	Point The point to which the arc is drawn.

Close command

Ends the current figure and creates a line that connects the current point to the starting point of the figure. This command creates a line-join (corner) between the last segment and the first segment of the figure.

Syntax

`Z`
- or -
`z`

Point syntax

Describes the x-coordinate and y-coordinate of a point. See also **Point**.

Syntax
x,y - or - *x y*

Term	Description
x	**Double** The x-coordinate of the point.
y	**Double** The y-coordinate of the point.

Additional notes

Instead of a standard numerical value, you can also use the following special values. These values are case sensitive.

- **Infinity**: Represents **PositiveInfinity**.
- **-Infinity**: Represents **NegativeInfinity**.
- **NaN**: Represents **NaN**.

Instead of using decimals or integers, you can use scientific notation. For example, `+1.e17` is a valid value.

Design tools that produce move and draw commands

Using the **Pen** tool and other drawing tools in Blend for Microsoft Visual Studio 2015 will usually produce a **Path** object, with move and draw commands.

You might see existing move and draw command data in some of the control parts defined in the Windows Runtime XAML default templates for controls. For example, some controls use a **PathIcon** that has the data defined as move and draw commands.

There are exporters or plug-ins available for other commonly used vector-graphics design tools that can output the vector in XAML form. These usually create **Path** objects in a layout container, with move and draw commands for **Path.Data**. There may be multiple **Path** elements in the XAML so that different brushes can be applied. Many of

these exporters or plug-ins were originally written for Windows Presentation Foundation (WPF) XAML or Silverlight, but the XAML path syntax is identical with Windows Runtime XAML. Usually, you can use chunks of XAML from an exporter and paste them right into a Windows Runtime XAML page. (However, you won't be able to use a **RadialGradientBrush**, if that was part of the converted XAML, because Windows Runtime XAML doesn't support that brush.)

Related topics

- Draw shapes
- Use brushes
- **Path.Data**
- **PathIcon**

XAML and whitespace

Article • 10/20/2022

Learn about the whitespace processing rules as used by XAML.

Whitespace processing

Consistent with XML, whitespace characters in XAML are space, linefeed, and tab. These correspond to the Unicode values 0020, 000A, and 0009 respectively. By default this whitespace normalization occurs when a XAML processor encounters any inner text found between elements in a XAML file:

- Linefeed characters between East Asian characters are removed.
- All whitespace characters (space, linefeed, tab) are converted into spaces.
- All consecutive spaces are deleted and replaced by one space.
- A space immediately following the start tag is deleted.
- A space immediately before the end tag is deleted.
- *East Asian characters* is defined as a set of Unicode character ranges U+20000 to U+2FFFD and U+30000 to U+3FFFD. This subset is also sometimes referred to as *CJK ideographs*. For more information, see http://www.unicode.org .

"Default" corresponds to the state denoted by the default value of the **xml:space** attribute.

Whitespace in inner text, and string primitives

The above normalization rules apply to inner text within XAML elements. After normalization, a XAML processor converts any inner text into an appropriate type like this:

- If the type of the property is not a collection, but is not directly an **Object** type, the XAML processor tries to convert to that type using its type converter. A failed conversion here results in a XAML parse error.
- If the type of the property is a collection, and the inner text is contiguous (no intervening element tags), the inner text is parsed as a single **String**. If the collection type cannot accept **String**, this also results in a XAML parser error.
- If the type of the property is **Object**, then the inner text is parsed as a single **String**. If there are intervening element tags, this results in a XAML parser error, because the **Object** type implies a single object (**String** or otherwise).

- If the type of the property is a collection, and the inner text is not contiguous, then the first substring is converted into a **String** and added as a collection item, the intervening element is added as a collection item, and finally the trailing substring (if any) is added to the collection as a third **String** item.

Whitespace and text content models

In practice, preserving whitespace is of concern only for a subset of all possible content models. That subset is composed of content models that can take a singleton **String** type in some form, a dedicated **String** collection, or a mixture of **String** and other types in lists, collections, or dictionaries.

Even for content models that can take strings, the default behavior within these content models is that any whitespace that remains is not treated as significant.

Preserving whitespace

Several techniques for preserving whitespace in the source XAML for eventual presentation are not affected by XAML processor whitespace normalization.

`xml:space="preserve"`: Specify this attribute at the level of the element where whitespace needs to be preserved. Note that this preserves all whitespace, including the spaces that might be added by code editors or design surfaces to align markup elements as a visually intuitive nesting. Whether those spaces render is again a matter of the content model for the containing element. We don't recommend that you specify `xml:space="preserve"` at the root level, because the majority of object models don't consider whitespace as significant one way or another. It is a better practice to only set the attribute specifically at the level of elements that render whitespace within strings, or are whitespace significant collections.

Entities and nonbreaking spaces: XAML supports placing any Unicode entity within a text object model. You can use dedicated entities such as nonbreaking space (in UTF-8 encoding). You can also use rich text controls that support nonbreaking space characters. Be cautious if you are using entities to simulate layout characteristics such as indents, because the run-time output of the entities vary based on a greater number of factors than would the general layout facilities, such as proper use of panels and margins.

XAML namescopes

Article • 10/20/2022

A *XAML namescope* stores relationships between the XAML-defined names of objects and their instance equivalents. This concept is similar to the wider meaning of the term *namescope* in other programming languages and technologies.

How XAML namescopes are defined

Names in XAML namescopes enable user code to reference the objects that were initially declared in XAML. The internal result of parsing XAML is that the runtime creates a set of objects that retain some or all of the relationships these objects had in the XAML declarations. These relationships are maintained as specific object properties of the created objects, or are exposed to utility methods in the programming model APIs.

The most typical use of a name in a XAML namescope is as a direct reference to an object instance, which is enabled by the markup compile pass as a project build action, combined with a generated **InitializeComponent** method in the partial class templates.

You can also use the utility method **FindName** yourself at run time to return a reference to objects that were defined with a name in the XAML markup.

More about build actions and XAML

What happens technically is that the XAML itself undergoes a markup compiler pass at the same time that the XAML and the partial class it defines for code-behind are compiled together. Each object element with a **Name** or x:Name attribute defined in the markup generates an internal field with a name that matches the XAML name. This field is initially empty. Then the class generates an **InitializeComponent** method that is called only after all the XAML is loaded. Within the **InitializeComponent** logic, each internal field is then populated with the **FindName** return value for the equivalent name string. You can observe this infrastructure for yourself by looking at the ".g" (generated) files that are created for each XAML page in the /obj subfolder of a Windows Runtime app project after compilation. You can also see the fields and **InitializeComponent** method as members of your resulting assemblies if you reflect over them or otherwise examine their interface language contents.

Note Specifically for Visual C++ component extensions (C++/CX) apps, a backing field for an **x:Name** reference is not created for the root element of a XAML file. If you need to reference the root object from C++/CX code-behind, use other APIs or tree traversal.

For example you can call **FindName** for a known named child element and then call **Parent**.

Creating objects at run time with XamlReader.Load

XAML can be also be used as the string input for the **XamlReader.Load** method, which acts analogously to the initial XAML source parse operation. **XamlReader.Load** creates a new disconnected tree of objects at run time. The disconnected tree can then be attached to some point on the main object tree. You must explicitly connect your created object tree, either by adding it to a content property collection such as **Children**, or by setting some other property that takes an object value (for example, loading a new **ImageBrush** for a **Fill** property value).

XAML namescope implications of XamlReader.Load

The preliminary XAML namescope defined by the new object tree created by **XamlReader.Load** evaluates any defined names in the provided XAML for uniqueness. If names in the provided XAML are not internally unique at this point, **XamlReader.Load** throws an exception. The disconnected object tree does not attempt to merge its XAML namescope with the main application XAML namescope, if or when it is connected to the main application object tree. After you connect the trees, your app has a unified object tree, but that tree has discrete XAML namescopes within it. The divisions occur at the connection points between objects, where you set some property to be the value returned from a **XamlReader.Load** call.

The complication of having discrete and disconnected XAML namescopes is that calls to the **FindName** method as well as direct managed object references no longer operate against a unified XAML namescope. Instead, the particular object that **FindName** is called on implies the scope, with the scope being the XAML namescope that the calling object is within. In the direct managed object reference case, the scope is implied by the class where the code exists. Typically, the code-behind for run-time interaction of a "page" of app content exists in the partial class that backs the root "page", and therefore the XAML namescope is the root XAML namescope.

If you call **FindName** to get a named object in the root XAML namescope, the method will not find the objects from a discrete XAML namescope created by **XamlReader.Load**. Conversely, if you call **FindName** from an object obtained from out of the discrete XAML namescope, the method will not find named objects in the root XAML namescope.

This discrete XAML namescope issue only affects finding objects by name in XAML namescopes when using the FindName call.

To get references to objects that are defined in a different XAML namescope, you can use several techniques:

- Walk the entire tree in discrete steps with Parent and/or collection properties that are known to exist in your object tree structure (such as the collection returned by Panel.Children).
- If you are calling from a discrete XAML namescope and want the root XAML namescope, it is always easy to get a reference to the main window currently displayed. You can get the visual root (the root XAML element, also known as the content source) from the current application window in one line of code with the call `Window.Current.Content`. You can then cast to FrameworkElement and call FindName from this scope.
- If you are calling from the root XAML namescope and want an object within a discrete XAML namescope, the best thing to do is to plan ahead in your code and retain a reference to the object that was returned by XamlReader.Load and then added to the main object tree. This object is now a valid object for calling FindName within the discrete XAML namescope. You could keep this object available as a global variable or otherwise pass it by using method parameters.
- You can avoid names and XAML namescope considerations entirely by examining the visual tree. The VisualTreeHelper API enables you to traverse the visual tree in terms of parent objects and child collections, based purely on position and index.

XAML namescopes in templates

Templates in XAML provide the ability to reuse and reapply content in a straightforward way, but templates might also include elements with names defined at the template level. That same template might be used multiple times in a page. For this reason, templates define their own XAML namescopes, independent of the containing page where the style or template is applied. Consider this example:

XML

```
<Page
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"   >
  <Page.Resources>
    <ControlTemplate x:Key="MyTemplate">
      ....
      <TextBlock x:Name="MyTextBlock" />
    </ControlTemplate>
  </Page.Resources>
```

```
  <StackPanel>
    <SomeControl Template="{StaticResource MyTemplate}" />
    <SomeControl Template="{StaticResource MyTemplate}" />
  </StackPanel>
</Page>
```

Here, the same template is applied to two different controls. If templates did not have discrete XAML namescopes, the "MyTextBlock" name used in the template would cause a name collision. Each instantiation of the template has its own XAML namescope, so in this example each instantiated template's XAML namescope would contain exactly one name. However, the root XAML namescope does not contain the name from either template.

Because of the separate XAML namescopes, finding named elements within a template from the scope of the page where the template is applied requires a different technique. Rather than calling **FindName** on some object in the object tree, you first obtain the object that has the template applied, and then call **GetTemplateChild**. If you are a control author and you are generating a convention where a particular named element in an applied template is the target for a behavior that is defined by the control itself, you can use the **GetTemplateChild** method from your control implementation code. The **GetTemplateChild** method is protected, so only the control author has access to it. Also, there are conventions that control authors should follow in order to name parts and template parts and report these as attribute values applied to the control class. This technique makes the names of important parts discoverable to control users who might wish to apply a different template, which would need to replace the named parts in order to maintain control functionality.

Related topics

- XAML overview
- x:Name attribute
- Quickstart: Control templates
- **XamlReader.Load**
- **FindName**

XAML namespaces and namespace mapping

Article • 01/04/2023

This topic explains the XML/XAML namespace (**xmlns**) mappings as found in the root element of most XAML files. It also describes how to produce similar mappings for custom types and assemblies.

How XAML namespaces relate to code definition and type libraries

Both in its general purpose and for its application to Windows Runtime app programming, XAML is used to declare objects, properties of those objects, and object-property relationships expressed as hierarchies. The objects you declare in XAML are backed by type libraries or other representations that are defined by other programming techniques and languages. These libraries might be:

- The built-in set of objects for the Windows Runtime. This is a fixed set of objects, and accessing these objects from XAML uses internal type-mapping and activation logic.
- Distributed libraries that are provided either by Microsoft or by third parties.
- Libraries that represent the definition of a third-party control that your app incorporates and your package redistributes.
- Your own library, which is part of your project and which holds some or all of your user code definitions.

Backing type info is associated with particular XAML namespace definitions. XAML frameworks such as the Windows Runtime can aggregate multiple assemblies and multiple code namespaces to map to a single XAML namespace. This enables the concept of a XAML vocabulary that covers a larger programming framework or technology. A XAML vocabulary can be quite extensive—for example, most of the XAML documented for Windows Runtime apps in this reference constitutes a single XAML vocabulary. A XAML vocabulary is also extensible: you extend it by adding types to the backing code definitions, making sure to include the types in code namespaces that are already used as mapped namespace sources for the XAML vocabulary.

A XAML processor can look up types and members from the backing assemblies associated with that XAML namespace when it creates a run-time object representation. This is why XAML is useful as a way to formalize and exchange definitions of object-

construction behavior, and why XAML is used as a UI definition technique for a UWP app.

XAML namespaces in typical XAML markup usage

A XAML file almost always declares a default XAML namespace in its root element. The default XAML namespace defines which elements you can declare without qualifying them by a prefix. For example, if you declare an element `<Balloon />`, a XAML parser will expect that an element **Balloon** exists and is valid in the default XAML namespace. In contrast, if **Balloon** is not in the defined default XAML namespace, you must instead qualify that element name with a prefix, for example `<party:Balloon />`. The prefix indicates that the element exists in a different XAML namespace than the default namespace, and you must map a XAML namespace to the prefix **party** before you can use this element. XAML namespaces apply to the specific element on which they are declared, and also to any element that is contained by that element in the XAML structure. For this reason, XAML namespaces are almost always declared on root elements of a XAML file to take advantage of this inheritance.

The default and XAML language XAML namespace declarations

Within the root element of most XAML files, there are two **xmlns** declarations. The first declaration maps a XAML namespace as the default:
`xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"`

This is the same XAML namespace identifier used in several predecessor Microsoft technologies that also use XAML as a UI definition markup format. The use of the same identifier is deliberate, and is helpful when you migrate previously defined UI to a Windows Runtime app using C++, C#, or Visual Basic.

The second declaration maps a separate XAML namespace for the XAML-defined language elements, mapping it (typically) to the "x:" prefix:
`xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"`

This **xmlns** value, and the "x:" prefix it is mapped to, is also identical to the definitions used in several predecessor Microsoft technologies that use XAML.

The relationship between these declarations is that XAML is a language definition, and the Windows Runtime is one implementation that uses XAML as a language and defines

a specific vocabulary where its types are referenced in XAML.

The XAML language specifies certain language elements, and each of these should be accessible through XAML processor implementations working against the XAML namespace. The "x:" mapping convention for the XAML language XAML namespace is followed by project templates, sample code, and the documentation for language features. The XAML language namespace defines several commonly used features that are necessary even for basic Windows Runtime apps using C++, C#, or Visual Basic. For example, to join any code-behind to a XAML file through a partial class, you must name that class as the x:Class attribute in the root element of the relevant XAML file. Or, any element as defined in a XAML page as a keyed resource in a ResourceDictionary and XAML resource references must have the x:Key attribute set on the object element in question.

Code namespaces that map to the default XAML namespace

The following is a list of code namespaces that are currently mapped to the default XAML namespace.

- Windows.UI
- Windows.UI.Xaml
- Windows.UI.Xaml.Automation
- Windows.UI.Xaml.Automation.Peers
- Windows.UI.Xaml.Automation.Provider
- Windows.UI.Xaml.Automation.Text
- Windows.UI.Xaml.Controls
- Windows.UI.Xaml.Controls.Primitives
- Windows.UI.Xaml.Data
- Windows.UI.Xaml.Documents
- Windows.UI.Xaml.Input
- Windows.UI.Xaml.Interop
- Windows.UI.Xaml.Markup
- Windows.UI.Xaml.Media
- Windows.UI.Xaml.Media.Animation
- Windows.UI.Xaml.Media.Imaging
- Windows.UI.Xaml.Media.Media3D
- Windows.UI.Xaml.Navigation
- Windows.UI.Xaml.Resources
- Windows.UI.Xaml.Shapes

- Windows.UI.Xaml.Threading
- Windows.UI.Text

Other XAML namespaces

In addition to the default namespace and the XAML language XAML namespace "x:", you may also see other mapped XAML namespaces in the initial default XAML for apps as generated by Microsoft Visual Studio.

d: (`http://schemas.microsoft.com/expression/blend/2008`)

The "d:" XAML namespace is intended for designer support, specifically designer support in the XAML design surfaces of Microsoft Visual Studio. The" d:" XAML namespace enables designer or design-time attributes on XAML elements. These designer attributes affect only the design aspects of how XAML behaves. The designer attributes are ignored when the same XAML is loaded by the Windows Runtime XAML parser when an app runs. Generally, the designer attributes are valid on any XAML element, but in practice there are only certain scenarios where applying a designer attribute yourself is appropriate. In particular, many of the designer attributes are intended to provide a better experience for interacting with data contexts and data sources while you are developing XAML and code that use data binding.

- **d:DesignHeight and d:DesignWidth attributes:** These attributes are sometimes applied to the root of a XAML file that Visual Studio or another XAML designer surface creates for you. For example, these attributes are set on the UserControl root of the XAML that is created if you add a new **UserControl** to your app project. These attributes make it easier to design the composition of the XAML content, so that you have some anticipation of the layout constraints that might exist once that XAML content is used for a control instance or other part of a larger UI page.

Note If you are migrating XAML from Microsoft Silverlight you might have these attributes on root elements that represent an entire UI page. You might want to remove the attributes in this case. Other features of the XAML designers such as the simulator are probably more useful for designing page layouts that handle scaling and view states well than is a fixed size page layout using **d:DesignHeight** and **d:DesignWidth**.

- **d:DataContext attribute:** You can set this attribute on a page root or a control to override any explicit or inherited DataContext that object otherwise has.
- **d:DesignSource attribute:** Specifies a design-time data source for a CollectionViewSource, overriding Source.

- **d:DesignInstance and d:DesignData markup extensions:** These markup extensions are used to provide the design-time data resources for either **d:DataContext** or **d:DesignSource**. We won't fully document how to use design-time data resources here. For more info, see Design-Time Attributes. For some usage examples, see Sample data on the design surface, and for prototyping.

mc: (`http://schemas.openxmlformats.org/markup-compatibility/2006`)

" mc:" indicates and supports a markup compatibility mode for reading XAML. Typically, the "d:" prefix is associated with the attribute **mc:Ignorable**. This technique enables run-time XAML parsers to ignore the design attributes in "d:".

local: and common:

"local:" is a prefix that is often mapped for you within the XAML pages for a templated UWP app project. It's mapped to refer to the same namespace that's created to contain the x:Class attribute and code for all the XAML files including app.xaml. So long as you define any custom classes you want to use in XAML in this same namespace, you can use the **local:** prefix to refer to your custom types in XAML. A related prefix that comes from a templated UWP app project is **common:**. This prefix refers to a nested "Common" namespace that contains utility classes such as converters and commands, and you can find the definitions in the Common folder in the **Solution Explorer** view.

vsm:

Do not use. "vsm:" is a prefix that is sometimes seen in older XAML templates imported from other Microsoft technologies. The namespace originally addressed a legacy namespace tooling issue. You should delete XAML namespace definitions for "vsm:" in any XAML you use for the Windows Runtime, and change any prefix usages for **VisualState**, **VisualStateGroup** and related objects to use the default XAML namespace instead. For more info on XAML migration, see Migrating Silverlight or WPF XAML/code to a Windows Runtime app.

Mapping custom types to XAML namespaces and prefixes

You can map a XAML namespace so that you can use XAML to access your own custom types. In other words, you are mapping a code namespace as it exists in a code

representation that defines the custom type, and assigning it a XAML namespace along with a prefix for usage. Custom types for XAML can be defined either in a Microsoft .NET language (C# or Microsoft Visual Basic) or in C++. The mapping is made by defining an **xmlns** prefix. For example, `xmlns:myTypes` defines a new XAML namespace that is accessed by prefixing all usages with the token `myTypes:`.

An **xmlns** definition includes a value as well as the prefix naming. The value is a string that goes inside quotation marks, following an equal sign. A common XML convention is to associate the XML namespace with a Uniform Resource Identifier (URI), so that there is a convention for uniqueness and identification. You also see this convention for the default XAML namespace and the XAML language XAML namespace, as well as for some lesser-used XAML namespaces that are used by Windows Runtime XAML. But for a XAML namespace that maps custom types, instead of specifying a URI, you begin the prefix definition with the token "using:". Following the "using:" token, you then name the code namespace.

For example, to map a "custom1" prefix that enables you to reference a "CustomClasses" namespace, and use classes from that namespace or assembly as object elements in XAML, your XAML page should include the following mapping on the root element: `xmlns:custom1="using:CustomClasses"`

Partial classes of the same page scope do not need to be mapped. For example, you don't need prefixes to reference any event handlers that you defined for handling events from the XAML UI definition of your page. Also, many of the starting XAML pages from Visual Studio generated projects for a Windows Runtime app using C++, C#, or Visual Basic already map a "local:" prefix, which references the project-specified default namespace and the namespace used by partial class definitions.

CLR language rules

If you are writing your backing code in a .NET language (C# or Microsoft Visual Basic), you might be using conventions that use a dot (".") as part of namespace names to create a conceptual hierarchy of code namespaces. If your namespace definition contains a dot, the dot should be part of the value you specify after the "using:" token.

If your code-behind file or code definition file is a C++ file, there are certain conventions that still follow the common language runtime (CLR) language form, so that there is no difference in the XAML syntax. If you declare nested namespaces in C++, the separator between the successive nested namespace strings should be "." rather than "::" when you specify the value that follows the "using:" token.

Don't use nested types (such as nesting an enumeration within a class) when you define your code for use with XAML. Nested types can't be evaluated. There's no way for the XAML parser to distinguish that a dot is part of the nested type name rather than part of the namespace name.

Custom types and assemblies

The name of the assembly that defines the backing types for a XAML namespace is not specified in the mapping. The logic for which assemblies are available is controlled at the app-definition level and is part of basic app deployment and security principles. Declare any assembly that you want included as a code-definition source for XAML as a dependent assembly in project settings. For more info, see Creating Windows Runtime components in C# and Visual Basic.

If you are referencing custom types from the primary app's application definition or page definitions, those types are available without further dependent assembly configuration, but you still must map the code namespace that contains those types. A common convention is to map the prefix "local" for the default code namespace of any given XAML page. This convention is often included in starting project templates for XAML projects.

Attached properties

If you are referencing attached properties, the owner-type portion of the attached property name must either be in the default XAML namespace or be prefixed. It's rare to prefix attributes separately from their elements but this is one case where it's sometimes required, particularly for a custom attached property. For more info, see Custom attached properties.

Related topics

- XAML overview
- XAML syntax guide
- Creating Windows Runtime components in C# and Visual Basic
- C#, VB, and C++ project templates for Windows Runtime apps
- Migrating Silverlight or WPF XAML/code to a Windows Runtime app

---

Feedback

Template settings classes

Article • 10/20/2022

Prerequisites

We assume that you can add controls to your UI, set their properties, and attach event handlers. For instructions for adding controls to your app, see Add controls and handle events. We also assume that you know the basics of how to define a custom template for a control by making a copy of the default template and editing it. For more info on this, see Quickstart: Control templates.

The scenario for TemplateSettings classes

TemplateSettings classes provide a set of properties that are used when you define a new control template for a control. The properties have values such as pixel measurements for the size of certain UI element parts. The values are sometimes calculated values that come from control logic that isn't typically easy to override or even access. Some of the properties are intended as **From** and **To** values that control transitions and animations of parts, and thus the relevant **TemplateSettings** properties come in pairs.

There are several **TemplateSettings** classes. All of them are in the **Windows.UI.Xaml.Controls.Primitives** namespace. Here's a list of the classes, and a link to the **TemplateSettings** property of the relevant control. This **TemplateSettings** property is how you access the **TemplateSettings** values for the control, and can establish template bindings to its properties:

- **ComboBoxTemplateSettings**: value of **ComboBox.TemplateSettings**
- **GridViewItemTemplateSettings**: value of **GridViewItem.TemplateSettings**
- **ListViewItemTemplateSettings**: value of **ListViewItem.TemplateSettings**
- **ProgressBarTemplateSettings**: value of **ProgressBar.TemplateSettings**
- **ProgressRingTemplateSettings**: value of **ProgressRing.TemplateSettings**
- **SettingsFlyoutTemplateSettings**: value of **SettingsFlyout.TemplateSettings**
- **ToggleSwitchTemplateSettings**: value of **ToggleSwitch.TemplateSettings**
- **ToolTipTemplateSettings**: value of **ToolTip.TemplateSettings**

TemplateSettings properties are always intended to be used in XAML, not code. They are read-only sub-properties of a read-only **TemplateSettings** property of a parent control. For an advanced custom control scenario, where you're creating a new **Control**-based class and thus can influence the control logic, consider defining a custom

TemplateSettings property on the control in order to communicate info that might be useful for anyone that is re-templating the control. As that property's read-only value, define a new **TemplateSettings** class related to your control that has read-only properties for each of the info items that are relevant for template measurements, animation positioning, and so on, and give callers the runtime instance of that class that's initialized using your control logic. **TemplateSettings** classes are derived from DependencyObject, so that the properties can use the dependency property system for property-changed callbacks. But the dependency property identifiers for the properties aren't exposed as public API, because the **TemplateSettings** properties are meant to be read-only to callers.

How to use TemplateSettings in a control template

Here's an example that comes from the starting default XAML control templates. This particular one is from the default template of ProgressRing:

XML

```
<Ellipse
    x:Name="E1"
    Style="{StaticResource ProgressRingEllipseStyle}"
    Width="{Binding RelativeSource={RelativeSource TemplatedParent},
        Path=TemplateSettings.EllipseDiameter}"
    Height="{Binding RelativeSource={RelativeSource TemplatedParent},
        Path=TemplateSettings.EllipseDiameter}"
    Margin="{Binding RelativeSource={RelativeSource TemplatedParent},
        Path=TemplateSettings.EllipseOffset}"
    Fill="{TemplateBinding Foreground}"/>
```

The full XAML for the ProgressRing template is hundreds of lines, so this is just a tiny excerpt. This XAML defines a control part that is one of 6 Ellipse elements that portray the spinning animation for indeterminate progress. As a developer, you might not like the circles and might use a different graphics primitive or a different basic shape for how the animation progresses. For example, you might compose a **ProgressRing** that uses a set of Rectangle elements arranged in a square instead. If so, each individual **Rectangle** component of your new template might look like this:

XML

```
<Rectangle
    x:Name="R1"
    Width="{Binding RelativeSource={RelativeSource TemplatedParent},
        Path=TemplateSettings.EllipseDiameter}"
```

```
    Height="{Binding RelativeSource={RelativeSource TemplatedParent},
        Path=TemplateSettings.EllipseDiameter}"
    Margin="{Binding RelativeSource={RelativeSource TemplatedParent},
        Path=TemplateSettings.EllipseOffset}"
    Fill="{TemplateBinding Foreground}"/>
```

The reason that the **TemplateSettings** properties are useful here is because they are calculated values coming from the basic control logic of ProgressRing. The calculation is dividing up the overall ActualWidth and ActualHeight of the **ProgressRing**, and allotting a calculated measurement for each of the motion elements in its templates so that the template parts can size to content.

Here's another example usage from the default XAML control templates, this time showing one of the property sets that are the **From** and **To** of an animation. This is from the ComboBox default template:

XML

```
<VisualStateGroup x:Name="DropDownStates">
    <VisualState x:Name="Opened">
        <Storyboard>
            <SplitOpenThemeAnimation
               OpenedTargetName="PopupBorder"
               ContentTargetName="ScrollViewer"
               ClosedTargetName="ContentPresenter"
               ContentTranslationOffset="0"
               OffsetFromCenter="{Binding RelativeSource={RelativeSource
TemplatedParent},
                 Path=TemplateSettings.DropDownOffset}"
               OpenedLength="{Binding RelativeSource={RelativeSource
TemplatedParent},
                 Path=TemplateSettings.DropDownOpenedHeight}"
               ClosedLength="{Binding RelativeSource={RelativeSource
TemplatedParent},
                 Path=TemplateSettings.DropDownClosedHeight}" />
        </Storyboard>
   </VisualState>
...
</VisualStateGroup>
```

Again there is lots of XAML in the template so we only show an excerpt. And this is only one of several states and theme animations that each use the same ComboBoxTemplateSettings properties. For ComboBox, use of the **ComboBoxTemplateSettings** values through bindings enforces that related animations in the template will stop and start at positions that are based on shared values, and thus transition smoothly.

Note When you do use **TemplateSettings** values as part of your control template, make sure you're setting properties that match the type of the value. If not, you might need to create a value converter for the binding so that the target type of the binding can be converted from a different source type of the **TemplateSettings** value. For more info, see **IValueConverter**.

Related topics

- Quickstart: Control templates

x:Class attribute

Article • 10/20/2022

Configures XAML compilation to join partial classes between markup and code-behind. The code partial class is defined in a separate code file, and the markup partial class is created by code generation during XAML compilation.

XAML attribute usage

syntax

```
<object x:Class="namespace.classname"...>
  ...
</object>
```

XAML values

Term	Description
namespace	Optional. Specifies a namespace that contains the partial class identified by *classname*. If *namespace* is specified, a dot (.) separates *namespace* and *classname*. If *namespace* is omitted, *classname* is assumed to have no namespace.
classname	Required. Specifies the name of the partial class that connects the loaded XAML and your code-behind for that XAML.

Remarks

x:Class can be declared as an attribute for any element that is the root of a XAML file/object tree and is being compiled by build actions, or for the **Application** root in the application definition of a compiled application. Declaring **x:Class** on any element other than a root node, and under any circumstances for a XAML file that is not compiled with the **Page** build action, results in a compile-time error.

The class used as **x:Class** cannot be a nested class.

The value of the **x:Class** attribute must be a string that specifies the fully qualified name of a class. You can omit namespace information so long as that is how the code-behind is structured also (your class definition starts at the class level). The code-behind file for

a page or application definition must be within a code file that is included as part of the project. The code-behind class must be public. The code-behind class must be partial.

CLR language rules

Although your code-behind file can be a C++ file, there are certain conventions that still follow the CLR language form, so that there is no difference in the XAML syntax. In particular, the separator between the namespace and classname components of any **x:Class** value is always a dot ("."), even though the separator between namespace and classname in the C++ code file associated with the XAML is "::". If you declare nested namespaces in C++, then the separator between the successive nested namespace strings should also be "." rather than "::" when you specify the *namespace* part of the **x:Class** value.

x:DefaultBindMode attribute

Article • 10/20/2022

In XAML markup, specifies a default mode for x:Bind.

x:DefaultBindMode is available starting in Windows 10, version 1607 (Anniversary Update), SDK version 14393.

XAML attribute usage

syntax

```
<object x:DefaultBindMode="OneTime \| OneWay \| TwoWay" .../>
```

Remarks

x:Bind has a default mode of **OneTime**. This was chosen for performance reasons, as using **OneWay** causes more code to be generated to hookup and handle change detection. You can use **x:DefaultBindMode** to change the default mode for x:Bind for a specific segment of the markup tree. The specified mode applies to any x:Bind expressions on that element and its children, that do not explicitly specify a mode as part of the binding.

Related topics

- x:Bind markup extension

x:DeferLoadStrategy attribute

Article • 10/20/2022

> **Important**
>
> Starting in Windows 10, version 1703 (Creators Update), **x:DeferLoadStrategy** is superseded by the **x:Load attribute**. Using `x:Load="False"` is equivalent to `x:DeferLoadStrategy="Lazy"`, but provides the ability to unload the UI if required. See the **x:Load attribute** for more info.

You can use **x:DeferLoadStrategy="Lazy"** to optimize the startup or tree creation performance of your XAML app. When you use **x:DeferLoadStrategy="Lazy"**, creation of an element and its children is delayed, which decreases startup time and memory costs. This is useful to reduce the costs of elements that are shown infrequently or conditionally. The element will be realized when it's referred to from code or VisualStateManager.

However, the tracking of deferred elements by the XAML framework adds about 600 bytes to the memory usage for each element affected. The larger the element tree you defer, the more startup time you'll save, but at the cost of a greater memory footprint. Therefore, it's possible to overuse this attribute to the extent that your performance decreases.

XAML attribute usage

syntax

```
<object x:DeferLoadStrategy="Lazy" .../>
```

Remarks

The restrictions for using **x:DeferLoadStrategy** are:

- You must define an x:Name for the element, as there needs to be a way to find the element later.
- You can only defer types that derive from **UIElement** or **FlyoutBase**.
- You cannot defer root elements in a **Page**, a **UserControl**, or a **DataTemplate**.
- You cannot defer elements in a **ResourceDictionary**.

- You cannot defer loose XAML loaded with **XamlReader.Load**.
- Moving a parent element will clear out any elements that have not been realized.

There are several different ways to realize the deferred elements:

- Call **FindName** with the name that you defined on the element.
- Call **GetTemplateChild** with the name that you defined on the element.
- In a **VisualState**, use a **Setter** or **Storyboard** animation that targets the deferred element.
- Target the deferred element in any **Storyboard**.
- Use a binding that targets the deferred element.

> NOTE: Once the instantiation of an element has started, it is created on the UI thread, so it could cause the UI to stutter if too much is created at once.

Once a deferred element is created in any of the ways listed previously, several things happen:

- The **Loaded** event on the element is raised.
- Any bindings on the element are evaluated.
- If you have registered to receive property change notifications on the property containing the deferred element(s), the notification is raised.

You can nest deferred elements, however they have to be realized from the outer-most element in. If you try to realize a child element before the parent has been realized, an exception is raised.

Typically, we recommend that you defer elements that are not viewable in the first frame. A good guideline for finding candidates to be deferred is to look for elements that are being created with collapsed **Visibility**. Also, UI that is triggered by user interaction is a good place to look for elements that you can defer.

Be wary of deferring elements in a **ListView**, as it will decrease your startup time, but could also decrease your panning performance depending on what you're creating. If you are looking to increase panning performance, see the {x:Bind} markup extension and x:Phase attribute documentation.

If the x:Phase attribute is used in conjunction with **x:DeferLoadStrategy** then, when an element or an element tree is realized, the bindings are applied up to and including the current phase. The phase specified for **x:Phase** does not affect or control the deferral of the element. When a list item is recycled as part of panning, realized elements behave in the same way as other active elements, and compiled bindings (**{x:Bind}** bindings) are processed using the same rules, including phasing.

A general guideline is to measure the performance of your app before and after to make sure you are getting the performance that you want.

Example

XML

```
<Grid x:Name="DeferredGrid" x:DeferLoadStrategy="Lazy">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="Auto" />
        <ColumnDefinition Width="Auto" />
    </Grid.ColumnDefinitions>

    <Rectangle Height="100" Width="100" Fill="#F65314" Margin="0,0,4,4" />
    <Rectangle Height="100" Width="100" Fill="#7CBB00" Grid.Column="1"
Margin="4,0,0,4" />
    <Rectangle Height="100" Width="100" Fill="#00A1F1" Grid.Row="1"
Margin="0,4,4,0" />
    <Rectangle Height="100" Width="100" Fill="#FFBB00" Grid.Row="1"
Grid.Column="1" Margin="4,4,0,0" />
</Grid>
<Button x:Name="RealizeElements" Content="Realize Elements"
Click="RealizeElements_Click"/>
```

C#

```
private void RealizeElements_Click(object sender, RoutedEventArgs e)
{
    // This will realize the deferred grid.
    this.FindName("DeferredGrid");
}
```

x:FieldModifier attribute

Article • 10/20/2022

Modifies XAML compilation behavior, such that fields for named object references are defined with **public** access rather than the **private** default behavior.

XAML attribute usage

syntax

```
<object x:FieldModifier="public".../>
```

Dependencies

x:Name attribute must also be provided on the same element.

Remarks

The value for the **x:FieldModifier** attribute will vary by programming language. Valid values are **private**, **public**, **protected**, **internal** or **friend**. For C#, Microsoft Visual Basic or Visual C++ component extensions (C++/CX), you can give the string value "public" or "Public"; the parser doesn't enforce case on this attribute value.

Private access is the default.

x:FieldModifier is only relevant for elements with an x:Name attribute, because that name is used to reference the field once it is public.

Note Windows Runtime XAML doesn't support **x:ClassModifier** or **x:Subclass**.

x:Key attribute

Article • 10/20/2022

Uniquely identifies elements that are created and referenced as resources, and which exist within a ResourceDictionary.

XAML attribute usage

syntax

```
<ResourceDictionary>
  <object x:Key="stringKeyValue".../>
</ResourceDictionary>
```

XAML attribute usage (implicit ResourceDictionary)

syntax

```
<object.Resources>
  <object x:Key="stringKeyValue".../>
</object.Resources>
```

XAML values

Term	Description
object	Any object that is shareable. See ResourceDictionary and XAML resource references.
stringKeyValue	A true string used as a key, which must conform to the *XamlName*> grammar. See "XamlName grammar" below.

XamlName grammar

The following is the normative grammar for a string that is used as a key in the Universal Windows Platform (UWP) XAML implementation:

syntax

```
XamlName ::= NameStartChar (NameChar)*
NameStartChar ::= LetterCharacter | '_'
NameChar ::= NameStartChar | DecimalDigit
LetterCharacter ::= ('a'-'z') | ('A'-'Z')
DecimalDigit ::= '0'-'9'
CombiningCharacter::= none
```

- Characters are restricted to the lower ASCII range, and more specifically to Roman alphabet uppercase and lowercase letters, digits, and the underscore (_) character.
- The Unicode character range is not supported.
- A name cannot begin with a digit.

Remarks

Child elements of a **ResourceDictionary** generally include an **x:Key** attribute that specifies a unique key value within that dictionary. Key uniqueness is enforced at load time by the XAML processor. Non-unique **x:Key** values will result in XAML parse exceptions. If requested by {StaticResource} markup extension, a non-resolved key will also result in XAML parse exceptions.

x:Key and x:Name are not identical concepts. **x:Key** is used exclusively in resource dictionaries. x:Name is used for all areas of XAML. A **FindName** call using a key value will not retrieve a keyed resource. Objects defined in a resource dictionary may have an **x:Key**, an **x:Name** or both. The key and name are not required to match.

Note that in the implicit syntax shown, the **ResourceDictionary** object is implicit in how the XAML processor produces a new object to populate a **Resources** collection.

The code equivalent of specifying **x:Key** is any operation that uses a key with the underlying **ResourceDictionary**. For example, an **x:Key** applied in markup for a resource is equivalent to the value of the *key* parameter of **Insert** when you add the resource to a **ResourceDictionary**.

An item in a resource dictionary can omit a value for **x:Key** if it is a targeted **Style** or **ControlTemplate**; in each of these cases the implicit key of the resource item is the **TargetType** value interpreted as a string. For more info, see Quickstart: styling controls and ResourceDictionary and XAML resource references.

x:Load attribute

Article • 11/18/2024

You can use **x:Load** to optimize the startup, visual tree creation, and memory usage of your XAML app. Using **x:Load** has a similar visual effect to **Visibility**, except that when the element is not loaded, its memory is released and internally a small placeholder is used to mark its place in the visual tree.

The UI element attributed with x:Load can be loaded and unloaded via code, or using an x:Bind expression. This is useful to reduce the costs of elements that are shown infrequently or conditionally. When you use x:Load on a container such as Grid or StackPanel, the container and all of its children are loaded or unloaded as a group.

The tracking of deferred elements by the XAML framework adds about 600 bytes to the memory usage for each element attributed with x:Load, to account for the placeholder. Therefore, it's possible to overuse this attribute to the extent that your performance actually decreases. We recommend that you only use it on elements that need to be hidden. If you use x:Load on a container, then the overhead is paid only for the element with the x:Load attribute.

> **ⓘ Important**
>
> The x:Load attribute is available starting in Windows 10, version 1703 (Creators Update). The min version targeted by your Visual Studio project must be *Windows 10 Creators Update (10.0, Build 15063)* in order to use x:Load.

XAML attribute usage

syntax

```
<object x:Load="True" .../>
<object x:Load="False" .../>
<object x:Load="{x:Bind Path.to.a.boolean, Mode=OneWay}" .../>
```

Loading Elements

There are several different ways to load the elements:

- Use an x:Bind expression to specify the load state. The expression should return **true** to load and **false** to unload the element.

- Call **FindName** with the name that you defined on the element.
- Call **GetTemplateChild** with the name that you defined on the element.
- In a **VisualState**, use a **Setter** or **Storyboard** animation that targets the x:Load element.
- Target the unloaded element in any **Storyboard**.

> NOTE: Once the instantiation of an element has started, it is created on the UI thread, so it could cause the UI to stutter if too much is created at once.

Once a deferred element is created in any of the ways listed previously, several things happen:

- The **Loaded** event on the element is raised.
- The field for x:Name is set.
- Any x:Bind bindings on the element are applied.
- If you have registered to receive property change notifications on the property containing the deferred element(s), the notification is raised.

Unloading elements

To unload an element:

- Use an x:Bind expression to specify the load state. The expression should return **true** to load and **false** to unload the element.
- In a Page or UserControl, call **UnloadObject** and pass in the object reference
- Call **Windows.UI.Xaml.Markup.XamlMarkupHelper.UnloadObject** and pass in the object reference

When an object is unloaded, it will be replaced in the tree with a placeholder. The object instance will remain in memory until all references have been released. The UnloadObject API on a Page/UserControl is designed to release the references held by codegen for x:Name and x:Bind. If you hold additional references in app code they will also need to be released.

When an element is unloaded, all state associated with the element will be discarded, so if using x:Load as an optimized version of Visibility, then ensure all state is applied via bindings, or is re-applied by code when the Loaded event is fired.

Restrictions

The restrictions for using **x:Load** are:

- You must define an x:Name for the element, as there needs to be a way to find the element later.
- You can only use x:Load on types that derive from **UIElement** or **FlyoutBase**.
- You cannot use x:Load on root elements in a **Page**, a **UserControl**, or a **DataTemplate**.
- You cannot use x:Load on elements in a **ResourceDictionary**.
- You cannot use x:Load on loose XAML loaded with **XamlReader.Load**.
- Moving a parent element will clear out any elements that have not been loaded.

Remarks

You can use x:Load on nested elements, however they have to be realized from the outer-most element in. If you try to realize a child element before the parent has been realized, an exception is raised.

Typically, we recommend that you defer elements that are not viewable in the first frame. A good guideline for finding candidates to be deferred is to look for elements that are being created with collapsed **Visibility**. Also, UI that is triggered by user interaction is a good place to look for elements that you can defer.

Be wary of deferring elements in a **ListView**, as it will decrease your startup time, but could also decrease your panning performance depending on what you're creating. If you are looking to increase panning performance, see the {x:Bind} markup extension and x:Phase attribute documentation.

If the x:Phase attribute is used in conjunction with **x:Load** then, when an element or an element tree is realized, the bindings are applied up to and including the current phase. The phase specified for **x:Phase** does affect or control the loading state of the element. When a list item is recycled as part of panning, realized elements will behave in the same way as other active elements, and compiled bindings (**{x:Bind}** bindings) are processed using the same rules, including phasing.

A general guideline is to measure the performance of your app before and after to make sure you are getting the performance that you want.

To minimize changes in behavior (aside from performance) when adding **x:Load** to an element, **x:Bind** bindings are calculated at their normal times, as if no elements used **x:Load**. For example, OneTime **x:Bind** bindings are calculated when the root element loads. If the element is not realized at the time the **x:Bind** binding is calculated, then the calculated value is saved and applied to the element when it loads. This behavior may be surprising if you expected **x:Bind** bindings to be calculated when the element is realized.

Example

XML

```
<StackPanel>
    <Grid x:Name="DeferredGrid" x:Load="False">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto" />
            <ColumnDefinition Width="Auto" />
        </Grid.ColumnDefinitions>

        <Rectangle Height="100" Width="100" Fill="Orange" Margin="0,0,4,4"/>
        <Rectangle Height="100" Width="100" Fill="Green" Grid.Column="1"
Margin="4,0,0,4"/>
        <Rectangle Height="100" Width="100" Fill="Blue" Grid.Row="1"
Margin="0,4,4,0"/>
        <Rectangle Height="100" Width="100" Fill="Gold" Grid.Row="1"
Grid.Column="1" Margin="4,4,0,0"
                   x:Name="one" x:Load="{x:Bind
(x:Boolean)CheckBox1.IsChecked, Mode=OneWay}"/>
        <Rectangle Height="100" Width="100" Fill="Silver" Grid.Row="1"
Grid.Column="1" Margin="4,4,0,0"
                   x:Name="two" x:Load="{x:Bind Not(CheckBox1.IsChecked),
Mode=OneWay}"/>
    </Grid>

    <Button Content="Load elements" Click="LoadElements_Click"/>
    <Button Content="Unload elements" Click="UnloadElements_Click"/>
    <CheckBox x:Name="CheckBox1" Content="Swap Elements" />
</StackPanel>
```

C#

```
// This is used by the bindings between the rectangles and check box.
private bool Not(bool? value) { return !(value==true); }

private void LoadElements_Click(object sender, RoutedEventArgs e)
{
    // This will load the deferred grid, but not the nested
    // rectangles that have x:Load attributes.
    this.FindName("DeferredGrid");
}

private void UnloadElements_Click(object sender, RoutedEventArgs e)
{
     // This will unload the grid and all its child elements.
```

```
        this.UnloadObject(DeferredGrid);
    }
```

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

x:Name attribute

Article • 10/20/2022

Uniquely identifies object elements for access to the instantiated object from code-behind or general code. Once applied to a backing programming model, **x:Name** can be considered equivalent to the variable holding an object reference, as returned by a constructor.

XAML attribute usage

syntax

```
<object x:Name="XAMLNameValue".../>
```

XAML values

Term	Description
XAMLNameValue	A string that conforms to the restrictions of the XamlName grammar.

XamlName grammar

The following is the normative grammar for a string that is used as a key in this XAML implementation:

syntax

```
XamlName ::= NameStartChar (NameChar)*
NameStartChar ::= LetterCharacter | '_'
NameChar ::= NameStartChar | DecimalDigit
LetterCharacter ::= ('a'-'z') | ('A'-'Z')
DecimalDigit ::= '0'-'9'
CombiningCharacter::= none
```

- Characters are restricted to the lower ASCII range, and more specifically to Roman alphabet uppercase and lowercase letters, digits, and the underscore (_) character.
- The Unicode character range is not supported.
- A name cannot begin with a digit. Some tool implementations prepend an underscore (_) to a string if the user supplies a digit as the initial character, or the tool autogenerates **x:Name** values based on other values that contain digits.

Remarks

The specified **x:Name** becomes the name of a field that is created in the underlying code when XAML is processed, and that field holds a reference to the object. The process of creating this field is performed by the MSBuild target steps, which also are responsible for joining the partial classes for a XAML file and its code-behind. This behavior is not necessarily XAML-language specified; it is the particular implementation that Universal Windows Platform (UWP) programming for XAML applies to use **x:Name** in its programming and application models.

Each defined **x:Name** must be unique within a XAML namescope. Generally, a XAML namescope is defined at the root element level of a loaded page and contains all elements under that element in a single XAML page. Additional XAML namescopes are defined by any control template or data template that is defined on that page. At run time, another XAML namescope is created for the root of the object tree that is created from an applied control template, and also by object trees created from a call to **XamlReader.Load**. For more info, see XAML namescopes.

Design tools often autogenerate **x:Name** values for elements when they are introduced to the design surface. The autogeneration scheme varies depending on which designer you are using, but a typical scheme is to generate a string that starts with the class name that backs the element, followed by an advancing integer. For example, if you introduce the first **Button** element to the designer, you might see that in the XAML this element has the **x:Name** attribute value of "Button1".

x:Name cannot be set in XAML property element syntax, or in code using **SetValue**. **x:Name** can only be set using XAML attribute syntax on elements.

Note Specifically for C++/CX apps, a backing field for an **x:Name** reference is not created for the root element of a XAML file or page. If you need to reference the root object from C++ code-behind, use other APIs or tree traversal. For example you can call **FindName** for a known named child element and then call **Parent**.

x:Name and other Name properties

Some types used in UWP XAML also have a property named **Name**. For example, **FrameworkElement.Name** and **TextElement.Name**.

If **Name** is available as a settable property on an element, **Name** and **x:Name** can be used interchangeably in XAML, but an error results if both attributes are specified on the same element. There are also cases where there's a **Name** property but it's read-only

(like **VisualState.Name**). If that's the case you always use **x:Name** to name that element in the XAML and the read-only **Name** exists for some less-common code scenario.

Note **FrameworkElement.Name** generally should not be used as a way to change values originally set by **x:Name**, although there are some scenarios that are exceptions to that general rule. In typical scenarios, the creation and definition of XAML namescopes is a XAML processor operation. Modifying **FrameworkElement.Name** at run time can result in an inconsistent XAML namescope / private field naming alignment, which is hard to keep track of in your code-behind.

x:Name and x:Key

x:Name can be applied as an attribute to elements within a **ResourceDictionary** to act as a substitute for the x:Key attribute. (It's a rule that all elements in a **ResourceDictionary** must have an x:Key or x:Name attribute.) This is common for Storyboarded animations. For more info, see section of ResourceDictionary and XAML resource references.

x:Phase attribute

Article • 10/20/2022

Use **x:Phase** with the {x:Bind} markup extension to render **ListView** and **GridView** items incrementally and improve the panning experience. **x:Phase** provides a declarative way of achieving the same effect as using the **ContainerContentChanging** event to manually control the rendering of list items. Also see Update ListView and GridView items incrementally.

XAML attribute usage

syntax

```
<object x:Phase="PhaseValue".../>
```

XAML values

Term	Description
PhaseValue	A number that indicates the phase in which the element will be processed. The default is 0.

Remarks

If a list is panned fast with touch, or using the mouse wheel, then depending on the complexity of the data template, the list may not be able to render items fast enough to keep up with the speed of scrolling. This is particularly true for a portable device with a power-efficient CPU such as a tablet.

Phasing enables incremental rendering of the data template so that the contents can be prioritized, and the most important elements rendered first. This enables the list to show some content for each item if panning fast, and will render more elements of each template as time permits.

Example

XML

```
<DataTemplate x:Key="PhasedFileTemplate" x:DataType="model:FileItem">
    <Grid Width="200" Height="80">
        <Grid.ColumnDefinitions>
           <ColumnDefinition Width="75" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>
        <Image Grid.RowSpan="4" Source="{x:Bind ImageData}" MaxWidth="70"
MaxHeight="70" x:Phase="3"/>
        <TextBlock Text="{x:Bind DisplayName}" Grid.Column="1"
FontSize="12"/>
        <TextBlock Text="{x:Bind prettyDate}"  Grid.Column="1"  Grid.Row="1"
FontSize="12" x:Phase="1"/>
        <TextBlock Text="{x:Bind prettyFileSize}"  Grid.Column="1"
 Grid.Row="2" FontSize="12" x:Phase="2"/>
        <TextBlock Text="{x:Bind prettyImageSize}"  Grid.Column="1"
 Grid.Row="3" FontSize="12" x:Phase="2"/>
    </Grid>
</DataTemplate>
```

The data template describes 4 phases:

1. Presents the DisplayName text block. All controls without a phase specified will be implicitly considered to be part of phase 0.
2. Shows the prettyDate text block.
3. Shows the prettyFileSize and prettyImageSize text blocks.
4. Shows the image.

Phasing is a feature of {x:Bind} that works with controls derived from **ListViewBase** and that incrementally processes the item template for data binding. When rendering list items, **ListViewBase** renders a single phase for all items in the view before moving onto the next phase. The rendering work is performed in time-sliced batches so that as the list is scrolled, the work required can be re-assessed, and not performed for items that are no longer visible.

The **x:Phase** attribute can be specified on any element in a data template that uses {x:Bind}. When an element has a phase other than 0, the element will be hidden from view (via **Opacity**, not **Visibility**) until that phase is processed and bindings are updated. When a **ListViewBase**-derived control is scrolled, it will recycle the item templates from items that are no longer on screen to render the newly visible items. UI elements within the template will retain their old values until they are data-bound again. Phasing causes

that data-binding step to be delayed, and therefore phasing needs to hide the UI elements in case they are stale.

Each UI element may have only one phase specified. If so, that will apply to all bindings on the element. If a phase is not specified, phase 0 is assumed.

Phase numbers do not need to be contiguous and are the same as the value of **ContainerContentChangingEventArgs.Phase**. The **ContainerContentChanging** event will be raised for each phase before the **x:Phase** bindings are processed.

Phasing only affects {x:Bind} bindings, not {Binding} bindings.

Phasing will only apply when the item template is rendered using a control that is aware of phasing. For Windows 10, that means **ListView** and **GridView**. Phasing will not apply to data templates used in other item controls, or for other scenarios such as **ContentTemplate** or **Hub** sections—in those cases, all the UI elements will be data bound at once.

x:Uid directive

Article • 10/20/2022

Provides a unique identifier for markup elements. For Universal Windows Platform (UWP) XAML, this unique identifier is used by XAML localization processes and tools, such as using resources from a .resw resource file.

XAML attribute usage

syntax

```
<object x:Uid="stringID".../>
```

XAML values

Term	Description
stringID	A string that uniquely identifies a XAML element in an app, and becomes part of the resource path in a resource file. See Remarks.

Remarks

Use **x:Uid** to identify an object element in your XAML. Typically this object element is an instance of a control class or other element that is displayed in a UI. The relationship between the string you use in **x:Uid** and the strings you use in a resources file is that the resource file strings are the **x:Uid** followed by a dot (.) and then by the name of a specific property of the element that's being localized. Consider this example:

syntax

```
<Button x:Uid="GoButton" Content="Go"/>
```

To specify content to replace the display text **Go**, you must specify a new resource that comes from a resource file. Your resource file should contain an entry for the resource named "GoButton.Content". **Content** in this case is a specific property that's inherited by the **Button** class. You might also provide localized values for other properties of this button, for example you could provide a resource-based value for "GoButton.FlowDirection". For more info on how to use **x:Uid** and resource files together, see Localize strings in your UI and app package manifest.

The validity of which strings can be used for an **x:Uid** value is controlled in a practical sense by which strings are legal as an identifier in a resource file and a resource path.

x:Uid is discrete from **x:Name** both because of the stated XAML localization scenario, and so that identifiers that are used for localization have no dependencies on the programming model implications of **x:Name**. Also, **x:Name** is governed by the XAML namescope concept, whereas uniqueness for **x:Uid** is controlled by the package resource index (PRI) system. For more info, see Resource Management System.

UWP XAML has somewhat different rules for **x:Uid** uniqueness than previous XAML-utilizing technologies used. For UWP XAML it is legal for the same **x:Uid** ID value to exist as a directive on multiple XAML elements. However, each such element must then share the same resolution logic when resolving the resources in a resource file. Also, all XAML files in a project share a single resource scope for purposes of **x:Uid** resolution, there is no concept of **x:Uid** scopes being aligned to individual XAML files.

In some cases you'll be using a resource path rather than built-in functionality of the package resource index (PRI) system. Any string used as an **x:Uid** value defines a resource path that begins with ms-resource:///Resources/ and includes the **x:Uid** string. The path is completed by the names of the properties you specify in a resources file or are otherwise targeting.

Don't put **x:Uid** on property elements, that isn't allowed in Windows Runtime XAML.

{x:Bind} markup extension

Article • 10/20/2022

Note For general info about using data binding in your app with **{x:Bind}** (and for an all-up comparison between **{x:Bind}** and **{Binding}**), see Data binding in depth.

The **{x:Bind}** markup extension—new for Windows 10—is an alternative to **{Binding}**. **{x:Bind}** runs in less time and less memory than **{Binding}** and supports better debugging.

At XAML compile time, **{x:Bind}** is converted into code that will get a value from a property on a data source, and set it on the property specified in markup. The binding object can optionally be configured to observe changes in the value of the data source property and refresh itself based on those changes (`Mode="OneWay"`). It can also optionally be configured to push changes in its own value back to the source property (`Mode="TwoWay"`).

The binding objects created by **{x:Bind}** and **{Binding}** are largely functionally equivalent. But **{x:Bind}** executes special-purpose code, which it generates at compile-time, and **{Binding}** uses general-purpose runtime object inspection. Consequently, **{x:Bind}** bindings (often referred-to as compiled bindings) have great performance, provide compile-time validation of your binding expressions, and support debugging by enabling you to set breakpoints in the code files that are generated as the partial class for your page. These files can be found in your `obj` folder, with names like (for C#) `<view name>.g.cs`.

> **Tip**
>
> **{x:Bind}** has a default mode of **OneTime**, unlike **{Binding}**, which has a default mode of **OneWay**. This was chosen for performance reasons, as using **OneWay** causes more code to be generated to hookup and handle change detection. You can explicitly specify a mode to use OneWay or TwoWay binding. You can also use x:DefaultBindMode to change the default mode for **{x:Bind}** for a specific segment of the markup tree. The specified mode applies to any **{x:Bind}** expressions on that element and its children, that do not explicitly specify a mode as part of the binding.

Sample apps that demonstrate {x:Bind}

- {x:Bind} sample
- QuizGame

- XAML Controls Gallery

XAML attribute usage

syntax

```
<object property="{x:Bind}" .../>
-or-
<object property="{x:Bind propertyPath}" .../>
-or-
<object property="{x:Bind bindingProperties}" .../>
-or-
<object property="{x:Bind propertyPath, bindingProperties}" .../>
-or-
<object property="{x:Bind pathToFunction.functionName(functionParameter1,
functionParameter2, ...), bindingProperties}" .../>
```

Expand table

Term	Description
propertyPath	A string that specifies the property path for the binding. More info is in the Property path section below.
bindingProperties	
propName=value[, *propName=value*]*	One or more binding properties that are specified using a name/value pair syntax.
propName	The string name of the property to set on the binding object. For example, "Converter".
value	The value to set the property to. The syntax of the argument depends on the property being set. Here's an example of a *propName=value* usage where the value is itself a markup extension: `Converter={StaticResource myConverterClass}`. For more info, see Properties that you can set with {x:Bind} section below.

Examples

XAML

```
<Page x:Class="QuizGame.View.HostView" ... >
    <Button Content="{x:Bind Path=ViewModel.NextButtonText, Mode=OneWay}"
... />
</Page>
```

This example XAML uses **{x:Bind}** with a **ListView.ItemTemplate** property. Note the declaration of an **x:DataType** value.

XAML

```
  <DataTemplate x:Key="SimpleItemTemplate"
x:DataType="data:SampleDataGroup">
    <StackPanel Orientation="Vertical" Height="50">
      <TextBlock Text="{x:Bind Title}"/>
      <TextBlock Text="{x:Bind Description}"/>
    </StackPanel>
  </DataTemplate>
```

Property path

PropertyPath sets the **Path** for an **{x:Bind}** expression. **Path** is a property path specifying the value of the property, sub-property, field, or method that you're binding to (the source). You can mention the name of the **Path** property explicitly: `{x:Bind Path=...}`. Or you can omit it: `{x:Bind ...}`.

Property path resolution

{x:Bind} does not use the **DataContext** as a default source—instead, it uses the page or user control itself. So it will look in the code-behind of your page or user control for properties, fields, and methods. To expose your view model to **{x:Bind}**, you will typically want to add new fields or properties to the code behind for your page or user control. Steps in a property path are delimited by dots (.), and you can include multiple delimiters to traverse successive sub-properties. Use the dot delimiter regardless of the programming language used to implement the object being bound to.

For example: in a page, **Text="{x:Bind Employee.FirstName}"** will look for an **Employee** member on the page and then a **FirstName** member on the object returned by **Employee**. If you are binding an items control to a property that contains an employee's dependents, your property path might be "Employee.Dependents", and the item template of the items control would take care of displaying the items in "Dependents".

For C++/CX, **{x:Bind}** cannot bind to private fields and properties in the page or data model – you will need to have a public property for it to be bindable. The surface area for binding needs to be exposed as CX classes/interfaces so that we can get the relevant metadata. The **[Bindable]** attribute should not be needed.

With **x:Bind**, you do not need to use **ElementName=xxx** as part of the binding expression. Instead, you can use the name of the element as the first part of the path for